Lezione 3 – Metodi, overloading, array

Sottoprogrammi, passaggio dei parametri, vettori e matrici

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

30 settembre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 2** (*15 min*)
   - uno di voi alla lavagna con `ContaCaratteri`, `Palindromo` o `Acronimo`: lo commentiamo insieme
2. **I metodi** (*20 min*)
   - anatomia e chiamata; variabili locali e blocchi; passaggio dei parametri (sempre per valore)
   - overloading: stesso nome, firme diverse; conversioni implicite
3. **Gli array** (*25 min*)
   - vettori: creazione, `length`, for potenziato, copia e confronto, array come parametri
   - riempimento; array di riferimenti (`String[]`); matrici
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: `Vettori`, una libreria di funzioni su array
   - *Insieme, passo passo*: `Matrici` – lettura, stampa, trasposta, somma
5. **Wrap-up e Homework 3** (*10 min*)

Anatomia di un metodo – Somma.java

```java
// modificatori  tipo   nome  (parametri formali)
public static double somma(double a, double b) {
    double c;           // variabile locale
    c = a + b;
    return c;           // del tipo dell'intestazione
}

// "procedura": tipo void, nessun valore restituito
public static void stampa(String et, double v) {
    System.out.printf("%s = %.2f%n", et, v);
}

public static void main(String[] args) {
    double ris = somma(3.5, 4);       // espressione
    stampa("somma", ris);             // istruzione
    stampa("doppia", somma(ris, ris));
}
```

In Java i sottoprogrammi si chiamano **metodi** e vivono sempre *dentro una classe*. Sono tutti **funzioni**: le "procedure" sono funzioni di tipo `void`.

Intestazione: modificatori (`public`, `static`), tipo del risultato, nome, lista dei parametri formali (anche vuota: `()`).

Corpo: un blocco { } con dichiarazioni, istruzioni e, se il tipo non è `void`, un `return` con un'espressione *di quel tipo*.

Chiamata: se il metodo restituisce un valore, la chiamata è un'*espressione* di quel tipo (si può assegnare, stampare, passare a un altro metodo); se è `void`, è un'*istruzione*.

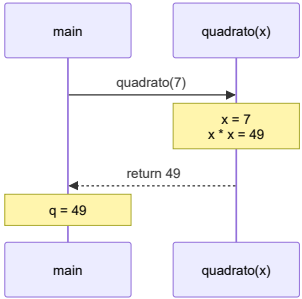
Chiamare un metodo: tre modi, e cosa succede

```java
public class Chiamate {

    public static int quadrato(int x) {
        return x * x;
    }

    public static void main(String[] args) {
        String s = "laboratorio";
        int n = s.length();          // 1. OGGETTO
        double r = Math.sqrt(2.0); // 2. CLASSE
        int q = quadrato(7);         // 3. nome da solo
    }
}
```

1. metodo applicato a un **oggetto** con il punto (dalla lezione 5)
2. metodo `static` di **un'altra classe**: `NomeClasse.metodo(...)`
3. metodo `static` della **stessa classe** in cui siamo (`quadrato` è definito in `Chiamate`, come `main`): basta il nome, come in C. Funziona anche `Chiamate.quadrato(7)`



Alla chiamata il controllo passa al metodo; al `return` torna al chiamante, insieme al valore.

Variabili locali e visibilità dei blocchi – `Visibilita.java`

```java
public static int raddoppia(int x) { // x: locale
    int y = x * 2;                   // y: locale
    // int x = 0;    // ERRORE: x esiste gia'
    x = 0;           // ok: assegna al parametro
    return y;
}

public static void main(String[] args) {
    int totale = 0;                  // tutto il main
    for (int i = 1; i <= 3; i++) {   // i: solo nel for
        int parziale = raddoppia(i); // idem
        totale += parziale;
    }
    // System.out.println(i);  // ERRORE: i non c'e'
    // System.out.println(y);  // ERRORE: y e' altrove
    System.out.println("totale = " + totale);   // 12
}
```

Una variabile dichiarata in un **blocco** { } esiste *solo* lì dentro: nasce alla dichiarazione, muore alla graffa di chiusura.

- i **parametri** sono variabili locali del metodo, inizializzate dalla chiamata
- `int x = 0;` è una **dichiarazione**: crea una nuova variabile, e `x` esiste già → errore di compilazione. `x = 0;` è un'**assegnazione**: cambia il valore del parametro, lecito (anche se qui inutile)
- la variabile di controllo di un `for` dichiarata nel `for` vive solo nel ciclo
- due metodi possono usare lo stesso nome per variabili diverse: non si vedono

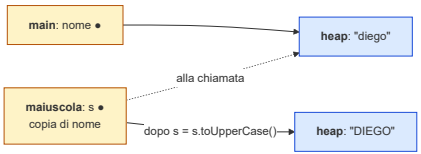
Il passaggio dei parametri: sempre per valore – PerValore.java

```java
public static void azzera(int i) {
    i = 0;                  // copia!
}

public static void maiuscola(String s) {
    s = s.toUpperCase();  // copia!
}

public static void main(String[] args) {
    int n = 30;
    azzera(n);
    System.out.println(n);       // 30

    String nome = new String("diego");
    maiuscola(nome);
    System.out.println(nome);    // diego
}
```



In Java il parametro formale riceve **sempre una copia** del parametro effettivo (niente scelta valore/riferimento come in C).

- **primitivo**: copia del valore. `azzera` azzera la sua `i`; `n` resta 30
- **riferimento**: copia del riferimento. Alla chiamata `s` è una seconda variabile nello stack che punta allo *stesso* oggetto `"diego"` di `nome`. `toUpperCase` crea `"DIEGO"` e `s = ...` fa puntare lì *solo la copia*: `nome` non è stato toccato
- con la copia del riferimento un metodo può però *modificare l'oggetto* puntato, se è modificabile: fra poco, con gli array

Overloading: stesso nome, firme diverse – `Overloading.java`

```java
public static int quadrato(int x) {
    return x * x;
}

public static double quadrato(double x) {
    return x * x;
}

// ERRORE: stessa firma di quadrato(int)
// public static double quadrato(int x)

quadrato(3)       // 9       -> quadrato(int)
quadrato(3.5)     // 12.25   -> quadrato(double)
```

Più metodi possono avere lo **stesso nome**, purché abbiano **liste di parametri diverse**: il compilatore sceglie in base agli argomenti della chiamata.

Firma (*signature*) di un metodo

nome + numero, tipo e ordine dei parametri.
Il **tipo restituito non ne fa parte**: due metodi che differiscono solo per quello non compilano.

Quando usarlo: la stessa operazione, concettualmente, su tipi diversi (`quadrato` di un intero o di un reale; `media` di un `int[]` o di un `double[]`: lo faremo oggi). Non per operazioni diverse che per caso hanno lo stesso nome.

Overloading e conversioni implicite

```java
public static void stampa(int x) {
    System.out.println("int: " + x);
}
public static void stampa(double x) {
    System.out.println("double: " + x);
}
public static void stampa(String s) {
    System.out.println("String: " + s);
}

stampa(42);        // int: 42
stampa(4.2);       // double: 4.2
stampa("42");      // String: 42
char c = 'A';
stampa(c);         // int: 65
stampa(3.14f);     // double: 3.14...
```

Se esiste un metodo con **esattamente** i tipi degli argomenti, vince quello. Altrimenti il compilatore cerca il metodo raggiungibile con le **conversioni implicite** (*widening*) dei tipi primitivi:

$$\texttt{char} \to \texttt{int} \to \texttt{double}$$

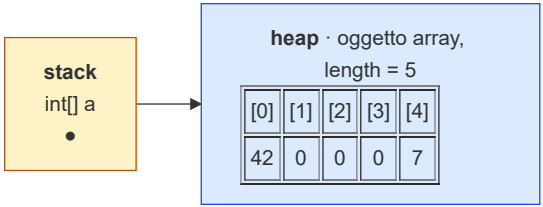
- `stampa('A')`: non c'è `stampa(char)`, il primo tipo raggiungibile è `int`
- senza `quadrato(double)`, `quadrato(3.5)` **non compila**: da `double` a `int` non si scende da soli

Se due metodi sono raggiungibili "allo stesso modo", la chiamata è **ambigua** e non compila. Le conversioni esplicite le vedremo la prossima lezione.

Che cos'è un array

Una sequenza di variabili **dello stesso tipo**, contigue in memoria, raggiunte tramite *un solo riferimento* e un **indice**.

```
int[] a = new int[5];    // 5 int, tutti a 0
a[0] = 42;               // primo elemento
a[4] = 7;                // ultimo: length - 1
// a[5] = 1;             // ferma il programma!
```



- gli array sono **oggetti**: si creano con `new`, vivono nello heap, la variabile è un riferimento
- gli elementi sono **inizializzati**: `0` per i numeri, `false` per i `boolean`, `null` per i riferimenti
- indici da `0` a `length - 1`; a differenza del C, l'accesso fuori dai limiti viene **sempre** rilevato (`ArrayIndexOutOfBoundsException`)
- la dimensione si decide alla creazione e **non cambia più**

`int[] a` dichiara il *riferimento*; `new int[5]` crea l'*oggetto* e ne restituisce l'indirizzo. `a.length` è il numero di elementi (è una variabile, non un metodo: niente parentesi).

Inizializzazione, `length` e i due `for` – `Vettore.java`

```java
int[] voti = {28, 30, 24, 27, 30};  // new implicito
System.out.println(voti.length);     // 5

for (int i = 0; i < voti.length; i++) { // indice
    System.out.printf("%d: %d%n", i, voti[i]);
}

double somma = 0;
for (int v : voti) {           // for potenziato
    somma += v;                // v: copia di voti[i]
}
System.out.println(somma / voti.length);   // 27.8

voti[4] = 18;
System.out.println(voti[voti.length - 1]); // 18
```

Inizializzazione con le graffe: il compilatore conta gli elementi e fa la `new` per voi.

`for` **potenziato** (*for-each*): `for (tipo x : array)` scorre *tutti* gli elementi, dal primo all'ultimo; `x` è una **copia** dell'elemento corrente.

Quale `for`?

- **potenziato**: devo solo *leggere* tutti gli elementi (somme, conteggi, ricerche)
- **classico**: mi serve l'*indice*, devo *modificare* gli elementi (`x = 0` non tocca l'array!), o non li scorro tutti

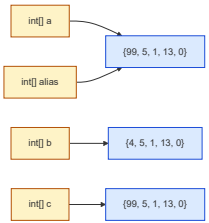
Copia e confronto: = e == lavorano sui riferimenti

```
int[] a = {4, 5, 1, 13, 0};

int[] alias = a;        // NON copia: stesso array
alias[0] = 99;
System.out.println(a[0]);  // 99 !

int[] b = new int[a.length];   // copia vera:
for (int i = 0; i < a.length; i++) {
    b[i] = a[i];         // elemento per elemento
}
b[0] = 4;
System.out.println(a[0]);  // 99: a non cambia

int[] c = {99, 5, 1, 13, 0};
System.out.println(a == alias);  // true
System.out.println(a == c);      // false !
```



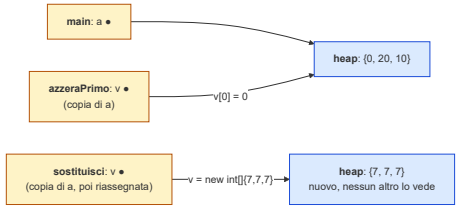
Esattamente come per le stringhe: = copia il riferimento, == confronta i riferimenti. a e c hanno lo stesso contenuto ma sono **due oggetti**.

Per copiare o confrontare il *contenuto* serve un ciclo, elemento per elemento (nell'esempio completo c'è anche il confronto). La prossima lezione vedremo che la libreria standard offre scorciatoie.

Array come parametri e come risultato – `ArrayParametro.java`

```java
public static void azzeraPrimo(int[] v) {
    v[0] = 0;                    // modifica
}
public static void sostituisci(int[] v) {
    v = new int[] {7, 7, 7};     // nessun effetto
}
public static int[] raddoppiato(int[] v) {    // crea
    int[] r = new int[v.length];
    for (int i = 0; i < v.length; i++) {
        r[i] = 2 * v[i];
    }
    return r;
}

public static void main(String[] args) {
    int[] a = {30, 20, 10};
    azzeraPrimo(a);
    System.out.println(a[0]);        // 0
    sostituisci(a);
    System.out.println(a[0]);        // 0
    int[] d = raddoppiato(a);
    System.out.println(d[1] + " " + a[1]);  // 40 20
}
```



Il parametro `v` è una *copia del riferimento*: punta allo **stesso array** del chiamante.

- `v[0] = 0` modifica l'oggetto condiviso: il chiamante **lo vede**
- `v = new int[]{...}` fa puntare la copia altrove: il chiamante **non se ne accorge**
- un metodo può **restituire** un array: crea l'oggetto, ne ritorna il riferimento. Il nome deve dire se *modifica* o *crea*

Dimensione massima e riempimento – `Riempimento.java`

```
final int MAX = 200;
int[] a = new int[MAX];    // dimensione MASSIMA
int n;                     // riempimento

System.out.print("Quanti elementi? ");
n = in.nextInt();
for (int i = 0; i < n; i++) {    // n elementi su MAX
    a[i] = in.nextInt();
}

int somma = 0;
for (int i = 0; i < n; i++) {    // NON a.length!
    somma += a[i];
}

// alternativa: dimensione decisa a run-time
int[] esatto = new int[n];    // n e' una variabile
```

La dimensione di un array è fissa, ma spesso quanti dati arriveranno *non si sa in anticipo*. Due strade:

Riempimento (come in C): array grande "a sufficienza" più una variabile `n` che dice quanti elementi sono *davvero* in uso. Da lì in poi ogni ciclo va da 0 a `n - 1`, **non** a `length - 1`.

Dimensione a tempo di esecuzione: `new int[n]` con `n` letto dall'utente. Funziona quando il numero di elementi si conosce *prima* di leggerli.

Il `for` potenziato scorre *tutto* l'array: con il riempimento non si può usare.

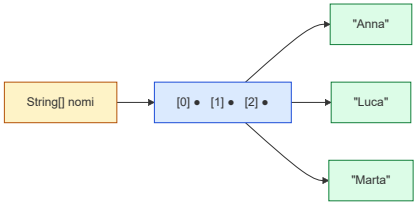
Array di riferimenti: `String[]`, cioè `args` – `ArrayDiStringhe.java`

```java
String[] nomi = new String[3];    // 3 null
System.out.println(nomi[0]);      // null
// nomi[0].length();  // NullPointerException

nomi[0] = "Anna";
nomi[1] = "Luca";
nomi[2] = "Marta";

String[] colori = {"rosso", "verde", "blu"};
for (String c : colori) {
    System.out.print(c.toUpperCase() + " ");
}

// args e' esattamente un String[]
for (int i = 0; i < args.length; i++) {
    System.out.println(i + ": " + args[i]);
}
```



Un array di tipo riferimento contiene **riferimenti**, non oggetti: dopo la `new` sono tutti `null`, e gli oggetti vanno creati (o assegnati) uno per uno.

Il `String[] args` del `main` è proprio questo: un array con una stringa per ogni parola scritta dopo il nome della classe. `java Prova uno due` → `args.length == 2`.

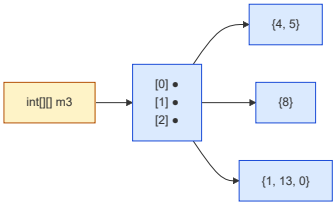
Array bidimensionali: matrici – `Matrice.java`

```java
int[][] m1 = new int[2][3];      // 2 x 3, tutti 0
int[][] m2 = {{4, 5, 9}, {1, 13, 0}}; // 2 x 3
int[][] m3 = {{4, 5}, {8}, {1, 13, 0}};

m1[1][2] = 7;                    // riga 1, colonna 2

for (int r = 0; r < m2.length; r++) {     // righe
    for (int c = 0; c < m2[r].length; c++) {
        System.out.printf("%4d", m2[r][c]);
    }
    System.out.println();
}

int[][] m4 = new int[2][];  // 2 righe
m4[0] = new int[5];
m4[1] = new int[3];
```



Una matrice in Java è un **array di array**: `m[r]` è la riga `r`, a sua volta un `int[]`.

- `m.length`: numero di righe; `m[r].length`: colonne della riga `r`
- le righe possono avere lunghezze **diverse** (array "frastagliati")
- si scorre con due `for` annidati: prima le righe, poi le colonne

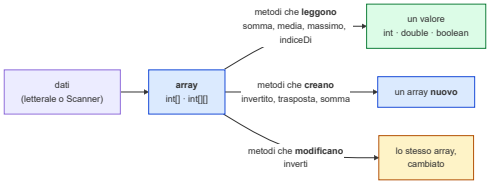
Commentiamo insieme (20 min) – il problema

Il problema. Molti programmi lavorano su sequenze di interi. Serve una piccola libreria di metodi su `int[]`, da usare in qualunque programma.

Le regole

- stampare un array nella forma `[12, 7, 3]`; calcolarne somma, massimo e media (anche di un `double[]`)
- cercare un valore: la posizione della sua *prima* occorrenza, oppure -1 se non c'è
- invertire un array: cambiando quello ricevuto, oppure restituendone uno nuovo, invertito
- massimo e media richiedono almeno un elemento

La soluzione è già scritta: `Vettori.java`. La leggiamo metodo per metodo, chiedendoci di quale tipo è ciascuno:



Commentiamo insieme – `Vettori.java` (1/3): i metodi che *leggono*

```java
public static void stampa(int[] v) {
    System.out.print("[");
    for (int i = 0; i < v.length; i++) {
        if (i > 0) {
            System.out.print(", ");
        }
        System.out.print(v[i]);
    }
    System.out.println("]");
}

public static int somma(int[] v) {
    int s = 0;
    for (int x : v) {
        s += x;
    }
    return s;
}
```

```java
public static int massimo(int[] v) {
    int max = v[0];
    for (int i = 1; i < v.length; i++) {
        if (v[i] > max) {
            max = v[i];
        }
    }
    return max;
}
```

Da notare, leggendo:

- `stampa`: serve il `for` classico per l'`if (i > 0)`, la virgola va *prima* di ogni elemento tranne il primo
- `somma`: solo lettura di tutti gli elementi → `for` potenziato
- `massimo`: parte da `v[0]` e da `i = 1`. Cosa succede con un array vuoto?
- nessuno dei tre tocca `v`: sono metodi che *leggono*

Commentiamo insieme – `Vettori.java` (2/3): overloading e ricerca

```java
/** Pre: v.length > 0 */
public static double media(int[] v) {
    double s = somma(v);    // double!
    return s / v.length;
}

/** Stesso nome, firma diversa */
public static double media(double[] v) {
    double s = 0;
    for (double x : v) {
        s += x;
    }
    return s / v.length;
}
```

`media(int[])` *riusa* `somma`; `media(double[])` non può: `somma` accetta solo `int[]`. Il compilatore sceglie l'una o l'altra guardando il tipo dell'argomento.

```java
/** Indice della prima occorrenza, o -1 */
public static int indiceDi(int[] v, int x) {
    for (int i = 0; i < v.length; i++) {
        if (v[i] == x) {
            return i;       // esce subito
        }
    }
    return -1;              // non trovato
}
```

Ricerca lineare: un `return` dentro il ciclo termina il metodo alla prima occorrenza; il `return -1` finale si raggiunge solo se il ciclo arriva in fondo.

Perché -1 e non 0? Perché 0 è un indice valido: il valore "non trovato" deve essere uno che *nessun* indice può assumere.

Commentiamo insieme – `Vettori.java` (3/3): modificare o creare

```java
/** Inverte SUL POSTO l'array del chiamante */
public static void inverti(int[] v) {
    for (int i = 0; i < v.length / 2; i++) {
        int tmp = v[i];
        v[i] = v[v.length - 1 - i];
        v[v.length - 1 - i] = tmp;
    }
}
```

Scambia `v[i]` con il simmetrico `v[length-1-i]` e si ferma a metà: `length / 2` è una divisione intera, con 5 elementi il centrale resta dov'è. Tipo `void`: l'effetto è *sull'argomento*.

```java
/** Restituisce un array NUOVO invertito */
public static int[] invertito(int[] v) {
    int[] r = new int[v.length];
    for (int i = 0; i < v.length; i++) {
        r[i] = v[v.length - 1 - i];
    }
    return r;
}
```

Stessa idea, ma scrive in un array `r` nuovo e lo restituisce: l'argomento resta intatto.

Due stili, una scelta

Modificare l'argomento (`inverti`) o restituire un nuovo array (`invertito`)? Entrambi sono corretti; il nome del metodo deve dire quale dei due fa, e il chiamante deve saperlo: dopo `inverti(v)` l'originale non c'è più.

Commentiamo insieme – eseguiamo e discutiamo

```
int[] v = {12, 7, 3, 21, 7, 9};
stampa(v);
System.out.println(somma(v));
System.out.println(media(v));
System.out.println(massimo(v));
System.out.println(indiceDi(v, 7));
System.out.println(indiceDi(v, 8));
int[] w = invertito(v);
stampa(w);
stampa(v);              // intatto
inverti(v);
stampa(v);              // modificato
double[] d = {1.5, 2.5, 3.0};
System.out.println(media(d));
```

```
[12, 7, 3, 21, 7, 9]
59
9.833333333333334
21
1
-1
[9, 7, 21, 3, 7, 12]
[12, 7, 3, 21, 7, 9]
[9, 7, 21, 3, 7, 12]
2.3333333333333335
```

Rispondiamo insieme:

1. `media(v)` e `media(d)`: quale dei due metodi viene chiamato, e chi lo decide?
2. `indiceDi(v, 7)` dà 1 e non 4: quale riga lo garantisce?
3. cosa succede con `massimo(new int[0])`? E con `media(new int[0])`?

Insieme, passo passo (30 min) – il problema

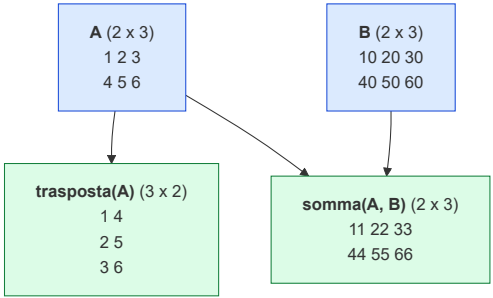
Il problema. Leggere due matrici di interi, stampare la prima, la sua trasposta e la somma delle due.

Le regole

- si leggono da tastiera il numero di righe e di colonne, poi gli elementi della prima matrice riga per riga, poi quelli della seconda: le due matrici hanno le stesse dimensioni
- la *trasposta* si ottiene scambiando righe e colonne: l'elemento in riga r e colonna c va in riga c e colonna r. Una matrice 2×3 diventa 3×2
- la *somma* si fa elemento per elemento
- le matrici si stampano una riga per riga, con i numeri incolonnati

Lo scriviamo da zero: `Matrici.java`.

Un esempio:



Insieme, passo passo – prima di scrivere codice

Prima di scrivere codice, decidiamo insieme:

1. quali **metodi**, e per ciascuno: *legge*, *modifica* o *crea*?
   → `leggi` (crea), `stampa` (legge), `trasposta` (crea), `somma` (crea), `stesseDimensioni` (legge)
2. le **firme**: cosa riceve e cosa restituisce ciascuno?
   → `int[][] trasposta(int[][] m)`
3. la trasposta di una $r \times c$ è $c \times r$: `t[c][r] = m[r][c]`
4. il `main`: legge le dimensioni, chiama `leggi` due volte, stampa i risultati

Da tenere a mente: chi controlla che le due matrici abbiano le stesse dimensioni, `somma` o il `main`?

Una possibile soluzione – `Matrici.java` (1/3): leggere e stampare

```java
/** Legge una matrice righe x colonne. */
public static int[][] leggi(Scanner in,
                            int righe, int colonne) {
    int[][] m = new int[righe][colonne];
    for (int r = 0; r < righe; r++) {
        for (int c = 0; c < colonne; c++) {
            m[r][c] = in.nextInt();
        }
    }
    return m;
}

public static void stampa(int[][] m) {
    for (int r = 0; r < m.length; r++) {
        for (int c = 0; c < m[r].length; c++) {
            System.out.printf("%4d", m[r][c]);
        }
        System.out.println();
    }
}
```

`leggi` riceve lo `Scanner` già aperto dal `main` (uno solo per tutto il programma), *crea* la matrice e la restituisce. Le dimensioni sono parametri: il metodo non chiede nulla all'utente, legge soltanto i numeri.

`stampa` *legge* e basta. Usa `m.length` e `m[r].length`, non le dimensioni: funziona con qualunque matrice, anche a righe di lunghezza diversa. `%4d` allinea le colonne.

Una possibile soluzione – `Matrici.java` (2/3): trasporre e sommare

```java
/** t[c][r] = m[r][c]: dimensioni scambiate */
public static int[][] trasposta(int[][] m) {
    int[][] t = new int[m[0].length][m.length];
    for (int r = 0; r < m.length; r++) {
        for (int c = 0; c < m[0].length; c++) {
            t[c][r] = m[r][c];
        }
    }
    return t;
}

/** Somma elemento per elemento. Pre: stesse dim. */
public static int[][] somma(int[][] a, int[][] b) {
    int[][] s = new int[a.length][a[0].length];
    for (int r = 0; r < a.length; r++) {
        for (int c = 0; c < a[r].length; c++) {
            s[r][c] = a[r][c] + b[r][c];
        }
    }
    return s;
}
```

`trasposta` *crea* `t` con le dimensioni **scambiate** (`m[0].length` righe, `m.length` colonne) e scambia gli indici nell'assegnazione. `m` non viene toccata.

`somma` *crea* `s` e si *fida* della precondizione scritta nel commento: non controlla nulla. Con dimensioni diverse `b[r][c]` uscirebbe dai limiti e il programma si fermerebbe. Il controllo lo fa il chiamante, prima di chiamare: stesso pattern di `media` e del suo `v.length > 0`.

Una possibile soluzione – `Matrici.java` (3/3): il `main`

```
public static boolean stesseDimensioni(int[][] a,
                                       int[][] b) {
    return a.length == b.length
        && a[0].length == b[0].length;
}

public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Righe e colonne: ");
    int righe = in.nextInt();
    int colonne = in.nextInt();
    int[][] a = leggi(in, righe, colonne);
    int[][] b = leggi(in, righe, colonne);
    in.close();

    stampa(a);
    stampa(trasposta(a));
    if (stesseDimensioni(a, b)) {
        stampa(somma(a, b));
    }
}
```

```
Righe e colonne: 2 3
1 2 3
4 5 6
10 20 30
40 50 60
   1   2   3
   4   5   6
   1   4
   2   5
   3   6
  11  22  33
  44  55  66
```

`stesseDimensioni` restituisce un `boolean`: si usa direttamente come condizione dell'`if`. Qui le dimensioni sono uguali per costruzione, ma il controllo costa poco e documenta l'intenzione.

`stampa(trasposta(a))`: il risultato di un metodo passato direttamente a un altro, senza variabile intermedia.

Riepilogo: cosa abbiamo trattato

Concetto (unità di riferimento)	**Dove compare oggi**
Intestazione, corpo, `return`, `void` (U7)	`Somma`; ogni metodo di `Vettori` e `Matrici`
Tre modi di chiamare un metodo (U7)	`Chiamate`; `media` che riusa `somma`
Variabili locali, parametri, blocchi (U7)	`Visibilita`; `tmp` in `inverti`
Passaggio per valore, riferimenti come parametri (U7)	`PerValore`, `ArrayParametro`; `inverti` / `invertito`
Overloading, firma, conversioni implicite (U7)	`Overloading`; le due `media`
Array: creazione, default, `length`, limiti (U9)	`Vettore`; `new int[righe][colonne]` in `leggi`
`for` potenziato: quando sì, quando no (U9)	`somma` (sì); `stampa`, `inverti` (no)
Copia e confronto elemento per elemento (U9)	`CopiaConfronto`; `invertito`
Dimensione massima e riempimento (U9)	`Riempimento`
Array di riferimenti, `args` (U9)	`ArrayDiStringhe`
Matrici, array di array, righe diverse (U9)	`Matrice`; `trasposta`, `somma`

Prossima lezione (L4): casting esplicito, `char` e Unicode, classi wrapper, la classe `Arrays` (le scorciatoie per copiare, confrontare, ordinare), argomenti a lunghezza variabile; e la programmazione modulare.

Homework 3 (da svolgere entro la prossima lezione)

1. `Statistiche.java`: legge quanti voti si vogliono inserire, li mette in un array e stampa minimo, massimo, media (2 decimali) e quanti voti sono sopra la media. Un metodo per ogni statistica: `minimo(int[])`, `massimo(int[])`, `media(int[])`, `quantiSopra(int[], double)`.
2. `Istogramma.java`: legge n cifre (da 0 a 9) in un array e stampa, per ogni cifra, quante volte compare, con una riga di asterischi. Un metodo `int[] contaCifre(int[] dati)` che restituisce un array di 10 conteggi (*il valore letto diventa un indice*) e un metodo `stampaIstogramma(int[] conteggi)`.
3. `Simmetrica.java`: legge n e una matrice $n \times n$; dice se è simmetrica (`m[r][c] == m[c][r]` per ogni `r`, `c`), stampa la traccia (la somma della diagonale) e la somma di ogni riga e di ogni colonna. Metodi `boolean isSimmetrica(int[][])`, `int traccia(int[][])`, `int[] sommeRighe(int[][])`, `int[] sommeColonne(int[][])`.

Per ogni metodo, decidete e scrivete in un commento se *legge*, *modifica* o *crea*. Usate solo ciò che abbiamo visto finora: niente `java.util.Arrays`, niente casting esplicito.

Domande?

mattia.fonisto@unina.it